Objetos y clases en Python

1. Qué es un objeto

Un **objeto** permite representar una entidad dentro de un programa, agrupando sus datos y las operaciones que puede realizar. Por ejemplo, una persona puede tener un nombre y una edad, y realizar acciones como saludar o cumplir años.

Los datos asociados a un objeto se llaman **atributos**. Las operaciones que definimos para trabajar con él se llaman **métodos**.

Elemento	Ejemplo en una persona
Atributos	nombre: "Ana" · edad: 20
Métodos	saludar() · cumplir_anios()

Los valores de sus atributos forman el **estado** del objeto. Si la persona cumple años, su edad cambia y, con ella, cambia su estado.

2. Qué es una clase

Una **clase** define cómo serán los objetos de un determinado tipo: qué atributos tendrán y qué métodos podremos utilizar con ellos. Cada objeto creado a partir de una clase se llama **instancia** de esa clase.

Por ejemplo, la clase **Persona** puede servir para crear un objeto que represente a Ana, de 20 años, y otro que represente a Luis, de 35. Comparten una definición, pero cada uno tiene sus propios datos.

3. Definir una clase y crear una instancia

Las clases se definen con la palabra reservada **class**. Por convención, sus nombres comienzan con mayúscula.

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

persona_1 = Persona("Ana", 20)
```

La última línea crea una instancia de **Persona**. La variable **persona_1** hace referencia a ese objeto. El método especial **__init__** se ejecuta automáticamente para inicializar sus atributos con los valores recibidos.

__init__ es un método especial, no una variable. Su función aquí es preparar el estado inicial de la instancia.

4. Qué significa self

self es el nombre convencional del primer parámetro de un método de instancia; no es una palabra reservada. Hace referencia al objeto y Python lo pasa automáticamente al llamar al método a través de él.

```
self.nombre = nombre
```

En esta instrucción, **nombre** es el parámetro que recibe el método y **self.nombre** es el atributo del objeto donde se guarda ese valor. Aunque suelen tener el mismo nombre, cumplen funciones distintas.

5. Acceder a los atributos y modificarlos

Para acceder a un atributo utilizamos el nombre del objeto seguido de un punto y del nombre del atributo.

```
persona_1 = Persona("Ana", 20)
persona_2 = Persona("Luis", 35)

print(persona_1.nombre)  # Ana
print(persona_2.edad)    # 35

persona_1.edad = 21

print(persona_1.edad)    # 21
print(persona_2.edad)    # 35
```

Cada instancia conserva sus propios valores. Modificar la edad de **persona_1** no cambia la edad de **persona_2**.

6. Definir y utilizar métodos

Los métodos son funciones definidas dentro de una clase. Ampliamos la clase anterior para que una persona pueda saludar y cumplir años:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        return f"Hola, me llamo {self.nombre}."

    def cumplir_anios(self):
        self.edad += 1

persona = Persona("Ana", 20)

print(persona.saludar())  # Hola, me llamo Ana.
persona.cumplir_anios()
print(persona.edad)       # 21
```

saludar() devuelve un mensaje. **cumplir_anios()** aumenta la edad del objeto en una unidad. En estas llamadas, **self** hace referencia a **persona**.

7. Métodos con parámetros y control del estado

Un método puede recibir datos, comprobar una condición y decidir si modifica el objeto. También puede devolver un resultado para que el resto del programa sepa qué ha ocurrido.

Representamos un producto mediante su nombre, su precio por unidad y las unidades disponibles. Partimos de un precio no negativo y de un stock entero no negativo; la cantidad que se intenta vender debe ser un número entero.

```
class Producto:
    def __init__(self, nombre, precio, stock):
        self.nombre = nombre
        self.precio = precio
        self.stock = stock

    def vender(self, cantidad):
        if cantidad <= 0 or cantidad > self.stock:
            return False
        self.stock -= cantidad
        return True

    def valor_stock(self):
        return self.precio * self.stock
```

cantidad es un parámetro de la llamada; **self.stock** es un atributo que conserva el objeto. Si la cantidad es válida, **vender()** descuenta las unidades y devuelve **True**. En caso contrario, devuelve **False** sin modificar el stock.

```
producto = Producto("Cuaderno", 3.5, 10)

print(producto.vender(3))      # True
print(producto.stock)          # 7
print(producto.valor_stock())  # 24.5

print(producto.vender(20))     # False
print(producto.stock)          # 7
```

La primera venta deja siete unidades. La segunda se rechaza porque supera el stock disponible. **return** termina la ejecución del método: al devolver **False**, Python no llega a ejecutar la resta.

valor_stock() calcula un resultado a partir de los atributos actuales, sin cambiarlos. No guardamos ese valor en otro atributo: lo calculamos cuando se necesita, para que refleje el stock del momento.

Devolver y mostrar son operaciones distintas. Los métodos usan **return** para entregar un valor al código que los llama. En el ejemplo, **print()** muestra ese valor, pero también podríamos guardarlo en una variable o usarlo en un **if**.

8. Herencia: ampliar una clase existente

La **herencia** permite definir una clase a partir de otra. La clase original se llama **clase base** y la nueva, **clase derivada** o **subclase**. La subclase puede reutilizar métodos de la base, añadir otros y dar una implementación propia a métodos existentes.

Partimos de la clase **Persona** del apartado 6. Un empleado es una persona y, además, tiene un puesto de trabajo. Podemos representar esa relación así:

```
class Empleado(Persona):
    def __init__(self, nombre, edad, puesto):
        super().__init__(nombre, edad)
        self.puesto = puesto

    def saludar(self):
        return f"Soy {self.nombre}, trabajo en {self.puesto}."

empleado = Empleado("Luis", 35, "ventas")

print(empleado.nombre)     # Luis
print(empleado.puesto)     # ventas
empleado.cumplir_anios()
print(empleado.edad)       # 36
print(empleado.saludar())  # Soy Luis, trabajo en ventas.
```

Empleado(Persona) indica que **Empleado** hereda de **Persona**. En este ejemplo, **super().__init__(nombre, edad)** llama al inicializador definido en **Persona**, que crea los atributos **nombre** y **edad** en el propio empleado. Después se añade **puesto**. No se crea una segunda persona dentro del objeto.

El método **cumplir_anios()** se utiliza sin volver a escribirlo: se hereda de **Persona**. En cambio, **Empleado** define su propia versión de **saludar()**. Esto se llama **sobrescritura de métodos**.

Si una subclase no define su propio **__init__**, puede utilizar el que hereda de la clase base. Si lo define y necesita la inicialización de la base, debe llamarla expresamente, como hemos hecho con **super()**.

```
print(isinstance(empleado, Empleado))  # True
print(isinstance(empleado, Persona))   # True
```

isinstance() permite comprobar si un objeto pertenece a una clase o a una de sus subclases. El empleado sigue siendo de tipo **Empleado**, pero también es una instancia de **Persona** por herencia.

Conviene usar herencia cuando existe una relación de especialización: un empleado **es una** persona. Compartir algunos datos o métodos no basta, por sí solo, para justificar esa relación.

8.1. Ampliar un método con super()

Sobrescribir un método permite darle un comportamiento propio en la subclase. Si queremos **aprovechar también el comportamiento de la base**, podemos llamarlo mediante **super()** desde nuestra nueva versión.

Partimos de **Persona** del apartado 6. En esta herencia sencilla, **super().saludar()** llama al **saludar()** de **Persona**:

```
class Alumno(Persona):
    def saludar(self):
        saludo = super().saludar()
        return saludo + " Soy alumno"

alumno = Alumno("Ana", 20)

print(alumno.saludar())
# Hola, me llamo Ana. Soy alumno
```

El saludo de **Persona** ya termina con un punto. Por eso añadimos **" Soy alumno"**, con un espacio inicial. Si el saludo de la base no tuviera punto final, añadiríamos **". Soy alumno"**, como en esa variante del ejemplo.

Qué ocurre al ejecutar alumno.saludar():
1. Python encuentra primero el método definido en **Alumno**.
2. super().saludar() obtiene el saludo de **Persona** para ese mismo alumno.
3. La variable local **saludo** guarda el texto devuelto.
4. return devuelve ese texto con la parte nueva añadida.

Alumno no define un __**init**__ propio, así que utiliza el de **Persona**. Ese inicializador crea **nombre** y **edad** en el objeto **alumno**. Todas las llamadas del ejemplo trabajan sobre ese mismo objeto.

Qué define la subclase	Qué sucede al llamar a saludar()
No define saludar()	Utiliza la versión heredada.
Define saludar() sin llamar a super()	Utiliza su propia versión, sin ejecutar automáticamente la de la base.
Define saludar() y llama a super().saludar()	Utiliza su propia versión y, dentro de ella, ejecuta también la versión que encuentra super().

La herencia la establece Alumno(Persona). super() no selecciona qué atributos heredar: permite invocar otro método siguiendo el orden de búsqueda. Aquí se usa con **saludar()**; también puede utilizarse con __**init**__**()** y otros métodos.

8.2. Herencia multinivel: una cadena de especializaciones

La **herencia multinivel** aparece cuando una clase hereda de otra que, a su vez, hereda de una tercera. Por ejemplo, un gerente es un empleado y un empleado es una persona:

```
Persona
    Empleado(Persona)
        Gerente(Empleado)
```

La indentación del esquema representa los niveles de herencia; no implica que las clases se definan unas dentro de otras. Partimos de **Persona** y **Empleado**, ya definidas, y añadimos **Gerente**:

```
class Gerente(Empleado):
    def __init__(self, nombre, edad, puesto, equipo):
        super().__init__(nombre, edad, puesto)
        self.equipo = equipo

    def describir_equipo(self):
        return f"{self.nombre} coordina a {self.equipo} personas."

gerente = Gerente("Elena", 42, "direccion", 5)

print(gerente.saludar())
# Soy Elena, trabajo en direccion.

gerente.cumplir_anios()
print(gerente.edad)               # 43
print(gerente.describir_equipo())
# Elena coordina a 5 personas.
```

Inicialización: Gerente.__init__ llama al de **Empleado**, y este llama al de **Persona**. Se inicializan **nombre**, **edad**, **puesto** y **equipo** en un único objeto, el gerente. En toda la cadena, **self** se refiere a ese mismo objeto.

Búsqueda de métodos: describir_equipo() está definido en **Gerente**; **saludar()** se encuentra en **Empleado**; y **cumplir_anios()** se encuentra en **Persona**. La búsqueda puede continuar por varios niveles, no solo por la base inmediata.

```
print(isinstance(gerente, Gerente))   # True
print(isinstance(gerente, Empleado))  # True
print(isinstance(gerente, Persona))   # True
```

Si **Gerente** definiera su propio **saludar()**, esa versión se encontraría antes que la de **Empleado**. Si no lo redefine, utiliza la versión heredada. En esta cadena simple, el orden de búsqueda es **Gerente, Empleado, Persona, object**.

8.3. Herencia múltiple: varias clases base

La **herencia múltiple** permite que una clase tenga más de una base directa. Los nombres se escriben entre paréntesis, separados por comas. Este ejemplo es independiente del anterior: una máquina multifunción ofrece las operaciones de una impresora y de un escáner.

```
class Impresora:
    def imprimir(self):
        return "Imprimiendo documento."

    def estado(self):
        return "Impresora preparada."

class Escaner:
    def escanear(self):
        return "Escaneando documento."

    def estado(self):
        return "Escaner preparado."

class Multifuncion(Impresora, Escaner):
    pass

maquina = Multifuncion()
print(maquina.imprimir())  # Imprimiendo documento.
print(maquina.escanear())  # Escaneando documento.
print(maquina.estado())    # Impresora preparada.
```

pass deja el cuerpo de la clase vacío. **Multifuncion** no añade métodos propios, pero puede utilizar los de ambas bases. También cumple **isinstance(maquina, Impresora)** e **isinstance(maquina, Escaner)**.

¿Qué ocurre si ambas bases tienen un método con el mismo nombre? Python utiliza un orden de búsqueda llamado **MRO** (Method Resolution Order). Podemos consultarlo con el atributo __**mro**__ de la clase:

```
print([clase.__name__ for clase in Multifuncion.__mro__])
# ['Multifuncion', 'Impresora', 'Escaner', 'object']
```

Para **maquina.estado()**, Python busca primero en **Multifuncion** y después en **Impresora**. Como allí encuentra el método, lo ejecuta. La llamada no ejecuta automáticamente los dos métodos llamados **estado()**.

En este ejemplo, cambiar las bases a **Multifuncion(Escaner, Impresora)** haría que se encontrara primero la versión de **Escaner**. En jerarquías más complejas, el orden completo depende también de los antepasados de cada base. Por eso conviene consultar el MRO en lugar de suponer un recorrido de izquierda a derecha por cada rama.

8.4. El caso del diamante y el significado de super()

Una jerarquía tiene forma de **diamante** cuando dos clases heredan de una misma base y otra clase hereda de las dos. En este ejemplo independiente, **Izquierda** y **Derecha** heredan de **Base**, y **Final** hereda de ambas:

```
class Base:
    def iniciar(self):
        print("Base")

class Izquierda(Base):
    def iniciar(self):
        print("Izquierda")
        super().iniciar()

class Derecha(Base):
    def iniciar(self):
        print("Derecha")
        super().iniciar()

class Final(Izquierda, Derecha):
    pass

Final().iniciar()
```

El orden de búsqueda es **Final, Izquierda, Derecha, Base, object**. El resultado es:

```
Izquierda
Derecha
Base
```

super() continúa la búsqueda después de la clase donde se utiliza, siguiendo el MRO del objeto actual. No significa siempre «llamar a mi padre directo».

1. Final no define **iniciar()**, así que se encuentra el de **Izquierda**.
2. El **super()** de **Izquierda** continúa por **Derecha**, aunque esta no sea su base directa.
3. El **super()** de **Derecha** continúa por **Base**.
4. Base termina la cadena: su método no hace otra llamada.

Aquí **Base.iniciar()** se ejecuta una sola vez. Es una **cadena cooperativa**: los métodos intermedios llaman a **super()** y aceptan argumentos compatibles. Si uno deja de llamar a **super()**, el recorrido se detiene en ese punto.

La misma regla se aplica a __**init**__: heredar de varias clases no ejecuta automáticamente todos sus inicializadores. Si varias bases necesitan inicialización, hay que diseñar sus llamadas y argumentos para que cooperen. Llamar directamente a cada base puede repetir la inicialización de un antepasado compartido.

En una cadena simple, el siguiente elemento suele ser la base directa. En herencia múltiple, puede ser otra rama. Python calcula el MRO manteniendo las prioridades de la jerarquía; si no puede obtener un orden coherente, produce **TypeError** al definir la clase.

9. Polimorfismo: utilizar objetos distintos de la misma manera

El **polimorfismo** permite que un mismo código utilice una operación común con objetos de distintos tipos. La llamada se escribe igual, pero cada objeto puede responder con un comportamiento diferente.

Utilizamos las clases **Persona** y **Empleado** ya definidas. Ambas ofrecen **saludar()**, por lo que una sola función puede presentar sus objetos:

```
def presentar(objeto):
    return objeto.saludar()

ana = Persona("Ana", 20)
luis = Empleado("Luis", 35, "ventas")

print(presentar(ana))   # Hola, me llamo Ana.
print(presentar(luis))  # Soy Luis, trabajo en ventas.
```

En la primera llamada, el parámetro **objeto** hace referencia a Ana y se ejecuta **Persona.saludar()**. En la segunda hace referencia a Luis y se ejecuta **Empleado.saludar()**. La función **presentar()** conserva exactamente la misma instrucción: **objeto.saludar()**.

La herencia y la sobrescritura explicadas antes permiten construir esas clases y sus métodos. Aquí el interés está en **utilizar sus objetos mediante una operación común**, sin añadir condiciones para distinguir sus tipos.

No es necesario compartir una clase base. En Python también puede participar una clase independiente si ofrece el comportamiento que necesita la función:

```
class Robot:
    def saludar(self):
        return "Sistema preparado."

print(presentar(Robot()))  # Sistema preparado.
```

Robot no hereda de **Persona**, pero la misma función lo puede presentar. Este uso de las operaciones disponibles en el objeto, sin exigir una clase concreta, se llama **duck typing**.

Para que los objetos sean compatibles en este ejemplo, **saludar()** debe poder llamarse sin argumentos adicionales y devolver una cadena con el saludo. Compartir solo el nombre del método no basta: también debe cumplir la operación esperada.

Si falta el método, la llamada produce **AttributeError**; si exige un argumento que no se proporciona, produce **TypeError**. Si únicamente imprime y no devuelve el texto, el resultado será **None**, que no cumple lo esperado aquí.

La utilidad es poder añadir nuevos tipos de objetos y seguir utilizando **presentar()** sin modificarla, siempre que respeten esa operación común.

10. Clases abstractas: definir una operación común

Una **clase abstracta** sirve como base para otras clases y declara operaciones que estas deben completar para poder crear objetos de ellas. Expresa qué operación debe estar disponible, aunque el cálculo concreto dependa de cada subclase.

Por ejemplo, podemos pedir el área de un rectángulo o de un cuadrado, pero la fórmula cambia. La clase **Figura** establece que sus objetos concretos deben ofrecer **area()**. Este ejemplo es independiente de las clases anteriores; utilizamos medidas positivas.

```
from abc import ABC, abstractmethod

class Figura(ABC):
    @abstractmethod
    def area(self):
        pass

    def describir(self):
        return f"Área: {self.area()}"

class Rectangulo(Figura):
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def area(self):
        return self.base * self.altura

rectangulo = Rectangulo(4, 3)
print(rectangulo.area())       # 12
print(rectangulo.describir())  # Área: 12
```

ABC y **abstractmethod** pertenecen al módulo **abc** de la biblioteca estándar: no hay que instalarlo. Heredar de **ABC** permite utilizar este mecanismo; @**abstractmethod** marca el método que debe tener una implementación concreta.

La línea que empieza por @ aplica un **decorador** al método definido debajo. Aquí indica a Python que **area()** es abstracto. No necesitamos conocer cómo se programan los decoradores para utilizarlo.

Rectangulo define su propio **area()** y ya no tiene métodos abstractos pendientes. Por eso podemos crear **Rectangulo(4, 3)**. El método **describir()** se hereda: al ejecutar **self.area()**, utiliza la fórmula del rectángulo.

Una clase abstracta puede contener **métodos ya implementados, atributos e incluso __init__**. No es necesario que todos sus métodos sean abstractos. En este ejemplo, **area()** es abstracto y **describir()** es un método concreto compartido.

10.1. Qué impide una clase abstracta y cuándo usarla

Python impide crear una instancia mientras queden métodos abstractos pendientes. La comprobación ocurre **al intentar crear el objeto**, no cuando se llama al método.

```
class FiguraIncompleta(Figura):
    pass

# Figura()             # TypeError: area sigue siendo abstracto
# FiguraIncompleta()   # TypeError: no ha implementado area
```

Las dos llamadas están comentadas para que el programa pueda continuar. Si se descomentan, producen **TypeError**. **FiguraIncompleta** hereda el método abstracto sin completarlo; definir la subclase está permitido, pero instanciarla todavía no.

En cambio, otra subclase que implemente el cálculo sí puede utilizarse:

```
class Cuadrado(Figura):
    def __init__(self, lado):
        self.lado = lado

    def area(self):
        return self.lado ** 2

print(Rectangulo(4, 3).describir())  # Área: 12
print(Cuadrado(5).describir())      # Área: 25
```

Ambos objetos utilizan el mismo **describir()** heredado, pero responden con su propia fórmula de **area()**. Así se conectan los tres conceptos: **herencia** para compartir la base, **abstracción** para exigir una operación y **polimorfismo** para utilizar objetos distintos mediante esa operación.

Escritura	Qué significa
pass	No realiza ninguna acción. Por sí solo no convierte un método en abstracto.
@abstractmethod	Marca un método abstracto dentro de una clase preparada con ABC.
Clase concreta	No conserva métodos abstractos pendientes y puede instanciarse.

Heredar de ABC por sí solo no bloquea la creación de objetos: deben quedar métodos marcados como abstractos. Asimismo, escribir **raise NotImplementedError** en un método normal no vuelve abstracta su clase; ese error aparecería al ejecutar el método.

Cuándo conviene: si varios tipos deben respetar una operación común y queremos detectar clases incompletas al instanciarlas. Por ejemplo, distintos exportadores pueden necesitar **exportar()**. No todas las clases base deben ser abstractas; el duck typing del apartado 9 sigue siendo otra forma válida de utilizar objetos compatibles.

Este mecanismo comprueba que no queden métodos abstractos pendientes. **No verifica que la fórmula, los argumentos o el resultado sean los correctos**: también hay que probar el comportamiento de las implementaciones.

Referencia: documentación de abc y PEP 3119.

11. En Python, todo es un objeto

Hasta aquí hemos creado nuestros propios objetos mediante una clase. Sin embargo, ya utilizábamos objetos antes de definir **Persona**: los números, las cadenas, las listas, las funciones y las propias clases son objetos en Python.

Cada objeto pertenece a un tipo. Podemos consultarlo con **type()**:

```
numero = 10
texto = "hola"
lista = [1, 2, 3]
persona = Persona("Ana", 20)

print(type(numero))           # <class 'int'>
print(type(texto))            # <class 'str'>
print(type(lista))            # <class 'list'>
print(type(persona).__name__) # Persona
```

Esto explica por qué las cadenas y las listas también tienen métodos:

```
print(texto.upper())  # HOLA

lista.append(4)
print(lista)          # [1, 2, 3, 4]
```

str y **list** son tipos que Python ya proporciona. Al definir **class Persona**, hemos creado un tipo propio, con atributos y métodos adecuados para nuestro programa.

12. F-strings: incluir valores dentro de un texto

La letra **f** delante de las comillas permite insertar valores dentro de una cadena. Python sustituye lo que aparece entre llaves **{ }** por su valor. Estas cadenas se llaman **f-strings**.

```
nombre = "Ana"
edad = 20

print(f"{nombre} tiene {edad} años.")
# Ana tiene 20 años.
```

En el ejemplo, **{nombre}** se sustituye por **Ana** y **{edad}** por **20**. Sin la **f**, las llaves y su contenido se muestran como texto:

```
print("Hola, {nombre}")   # Hola, {nombre}
print(f"Hola, {nombre}")  # Hola, Ana
```

La **f** pertenece a la cadena, no a **print()**: podemos guardar el texto con **mensaje = f"Hola, {nombre}"**. Dentro de las llaves también podemos utilizar operaciones, como **{edad + 1}**, o atributos de un objeto, como **{persona.edad}**. Los nombres utilizados deben estar definidos.

13. Try y except: gestionar un error previsto

En Python se escribe **try / except**; **catch** es el nombre utilizado en otros lenguajes. El bloque **try** contiene las instrucciones que intentamos ejecutar. El bloque **except** indica qué hacer si aparece el tipo de error señalado.

```
try:
    edad = int(input("Escribe tu edad: "))
    print(f"Has indicado {edad} años.")
except ValueError:
    print("Debes escribir un número entero.")

print("El programa continúa.")
```

Si escribimos 20: int() convierte el texto en un entero y se muestra **Has indicado 20 años.** El bloque **except** no se ejecuta.

Si escribimos veinte: la conversión produce **ValueError**. Python omite el resto del **try** y ejecuta el **except ValueError**, que muestra el mensaje. Después continúa con la instrucción situada fuera de ambos bloques.

try / except no corrige el dato ni vuelve a pedirlo automáticamente. Para repetir la pregunta necesitaríamos un bucle. Además, este **except** solo gestiona **ValueError**: otros tipos de error no quedan cubiertos por él.